.NET C# IT BOOTCAMP TEST 2026/2027

Binus Point of Sale (POS) Web App: BeeMart

BeeMart System adalah proyek aplikasi dari IT Division yang bertujuan untuk membangun platform Sistem Kasir terintegrasi menggunakan bahasa C# pada framework .NET 8. Proyek ini dirancang menggunakan arsitektur Separation of Concerns, di mana sistem dipisah menjadi dua proyek utama: WebClient (Frontend/UI) dan Backend API.

WebClient hanya bertindak sebagai consumer (pengonsumsi) data. Seluruh proses bisnis, logika aplikasi, dan persistensi/penyimpanan data dikelola secara terpusat di Backend API.

Alur Bisnis Inti Proyek ini mensimulasikan alur bisnis inti sebuah sistem Point of Sale (POS). Pengguna dapat:

- Mendaftar (Register) dan masuk (Login) dengan pembagian hak akses (Role) sebagai **Admin** atau **Kasir**.
- Melakukan transaksi baru (New Transaction) yang dilengkapi jam *real-time*, pencarian barang, input *quantity*, pemilihan metode pembayaran (Cash, Debit, Credit), dan kalkulasi subtotal serta total harga.
- Mengenerate *Invoice Number* secara otomatis pada setiap transaksi dengan format baku: [5 digit userID][ddmmyy][3 digit running number] yang tidak dapat diedit oleh *user*.
- Melihat Riwayat Transaksi (History Transaction) berdasarkan filter tanggal tertentu dengan pengurutan (Order By) *Ascending* berdasarkan Tanggal dan Jam transaksi.
- Mengakses data secara spesifik sesuai peran: Kasir hanya melihat data transaksinya sendiri, sementara Admin dapat melihat data seluruh Kasir.

Tujuan Pembelajaran

1. **Arsitektur Sistem**: Mempraktikkan arsitektur Separation of Concerns (Pemisahan antara UI dan API).
2. **Integrasi API**: Mendesain dan mengonsumsi RESTful API untuk fitur autentikasi, master produk, pembuatan transaksi, dan pelaporan riwayat transaksi.
3. **State Management**: Mengelola state pada sisi WebClient (seperti data kasir yang *login*, state keranjang belanja/cart, dan state pencarian barang).
4. **Keamanan & Validasi**: Menerapkan validasi input data di sisi klien maupun server, serta menerapkan kontrol akses otorisasi (*Role-based Authorization*).

5. **Desain Database**: Merancang dan mengelola relasi data (Entity Relationship Diagram / ERD) menggunakan relasi tabel *User*, *Product*, *Transaction*, dan *TransactionDetail*.

Aturan Teknis

- **Platform**: Wajib menggunakan ASP.NET Core C# dengan Framework .NET 8.
- **Frontend (WebClient)**: Gunakan HTML/JS Murni, ASP.NET Core Razor Pages, atau MVC. UI **wajib** dibuat dengan *mockup requirement*. Catatan: Tidak diperbolehkan menggunakan framework CSS Bootstrap. Frontend dilarang keras memiliki akses langsung ke database; seluruh data harus didapat melalui HTTP Request ke Backend API.
- **Backend API**: Gunakan ASP.NET Core API dengan .NET 8 (berbasis Controller atau Minimal API). Gunakan Entity Framework (EF) Core terhubung ke **SQL Server** sebagai ORM.
- **Standar API & Pagination**: Gunakan format komunikasi JSON dan terapkan endpoint versioning. Endpoint Riwayat Transaksi wajib menerapkan *Pagination* (maksimal **20 data** per halaman).
- **Error Handling**: Tampilkan penanganan error yang informatif kepada pengguna menggunakan UI Popup Message (misal: Toast / Alert / Windows Alert) atau Label Message.
- **I18N/Format**: Gunakan format lokal Indonesia untuk tampilan mata uang Rupiah (IDR) di seluruh UI dan pastikan waktu disesuaikan dengan zona waktu lokal.

Kriteria Penerimaan (Acceptance Criteria)

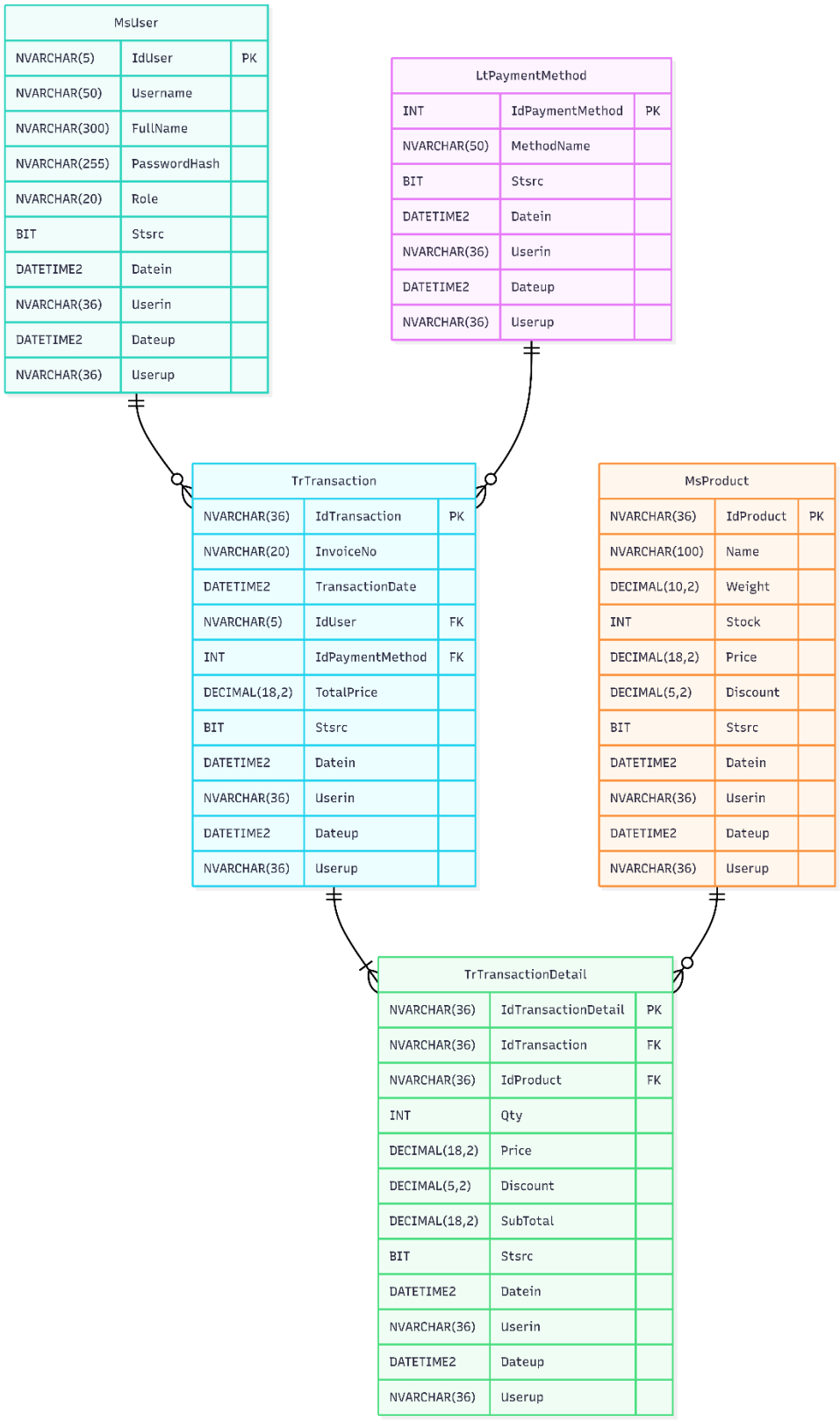
- Seluruh halaman (Login, Register, New Transaction, History Transaction) tersedia dan fungsional sesuai dengan deskripsi spesifikasi.
- WebClient 100% bergantung pada API (tidak ada koneksi langsung ke database dari WebClient).
- Autentikasi dan otorisasi berjalan dengan baik (endpoint riwayat transaksi secara akurat memisahkan hak akses antara Admin dan Kasir).
- Nomor Invoice ter-generate otomatis dengan format yang tepat dan berurutan.

📦 **Instruksi Pengumpulan Proyek** 📦 Mahasiswa WAJIB mengumpulkan file proyek secara lengkap yang mencakup empat komponen di bawah ini (termasuk folder aset gambar, file appsettings.json, file .sln, dan .csproj) dalam bentuk arsip (ZIP/RAR):

- **Folder Frontend (UI/WebClient)** ✅
- **Folder Backend (API)** ✅
- **Query SQL (Script Database)** ✅ - Sertakan script .sql yang berisi Query Create Table (DDL) dan Query Insert Data (DML/Seeding produk dan user) agar aplikasi bisa langsung diuji coba.

❌ **Peringatan Penting:** Segala bentuk kecurangan, plagiarisme, atau pelanggaran aturan teknis (seperti melakukan akses database langsung dari WebClient) akan mengakibatkan diskualifikasi dan tidak mendapatkan nilai. ❌

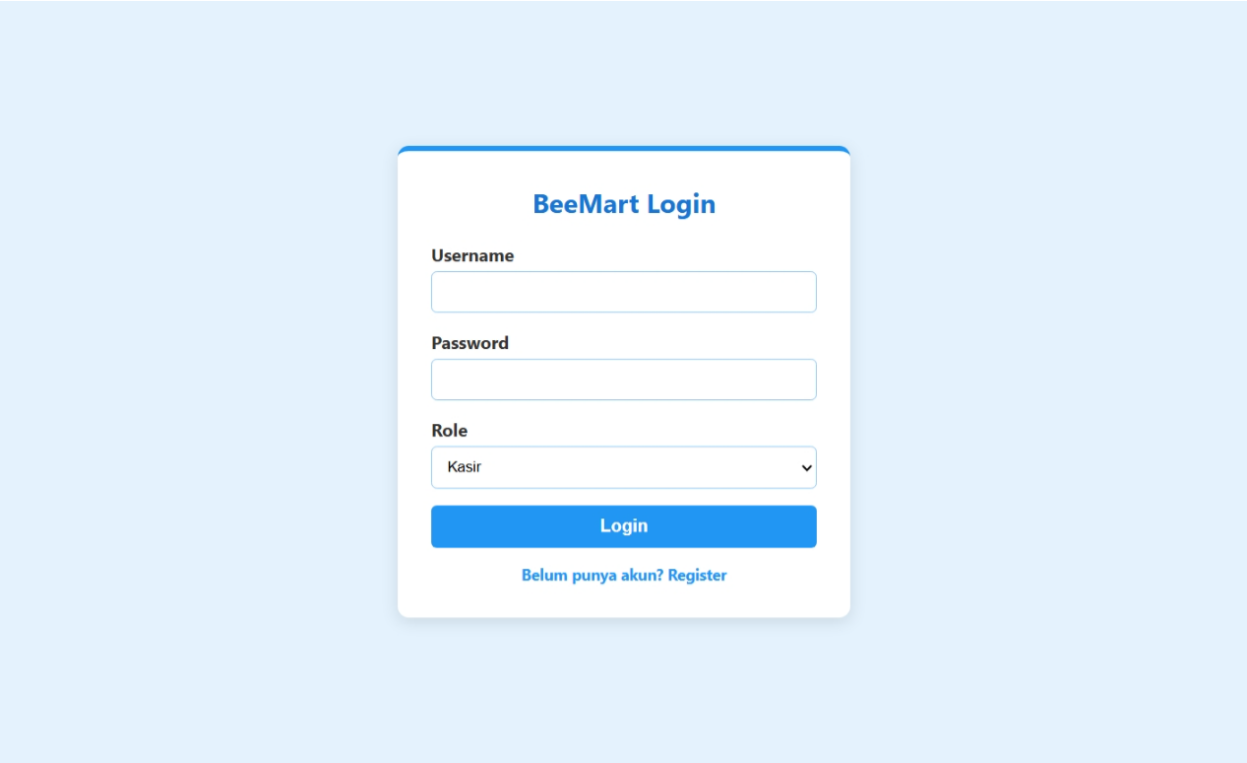
ERD Database



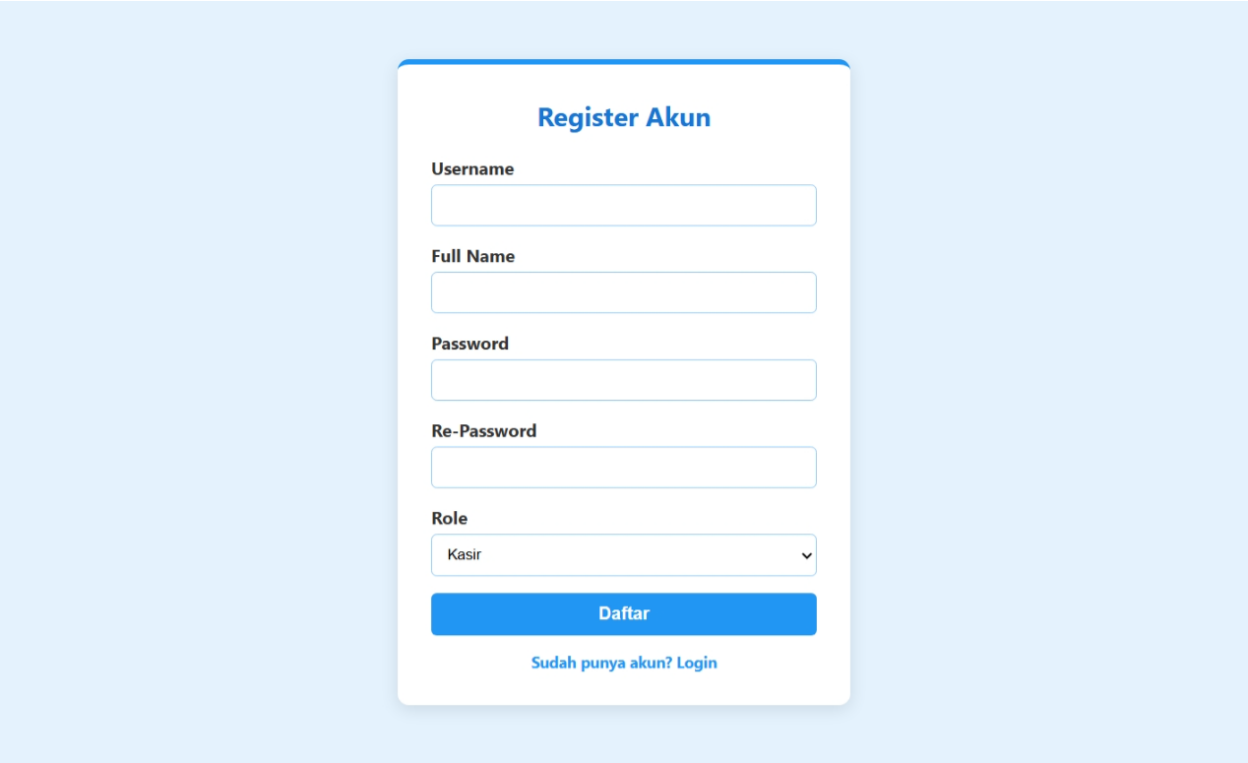
Berikut adalah detail ruang lingkup fitur yang harus diimplementasikan:

1. Login
   Halaman ini berfungsi sebagai gerbang utama untuk proses autentikasi dan otorisasi pengguna sebelum mereka dapat mengakses fitur-fitur di dalam aplikasi.

   Validasi yang harus diimplementasikan:
   a. *Username* dan *Password* wajib diisi.
   b. Kombinasi *Username*, *Password*, dan *Role* harus divalidasi dengan data pada tabel MsUser.
   c. Pencocokan *password* harus menggunakan metode *hashing*. *Password* yang disimpan di dalam *database* tidak boleh berupa teks murni (*plain text*).
   d. Jika *username*, *password*, atau *role* tidak sesuai, sistem harus mengembalikan pesan *error* "Invalid login".
   e. *Dropdown Role* harus memiliki pilihan data sebagai berikut:
      i. Name: "Admin", Value: "Admin"
      ii. Name: "Kasir", Value: "Cashier"



2. Register



Validasi yang harus diimplementasikan:

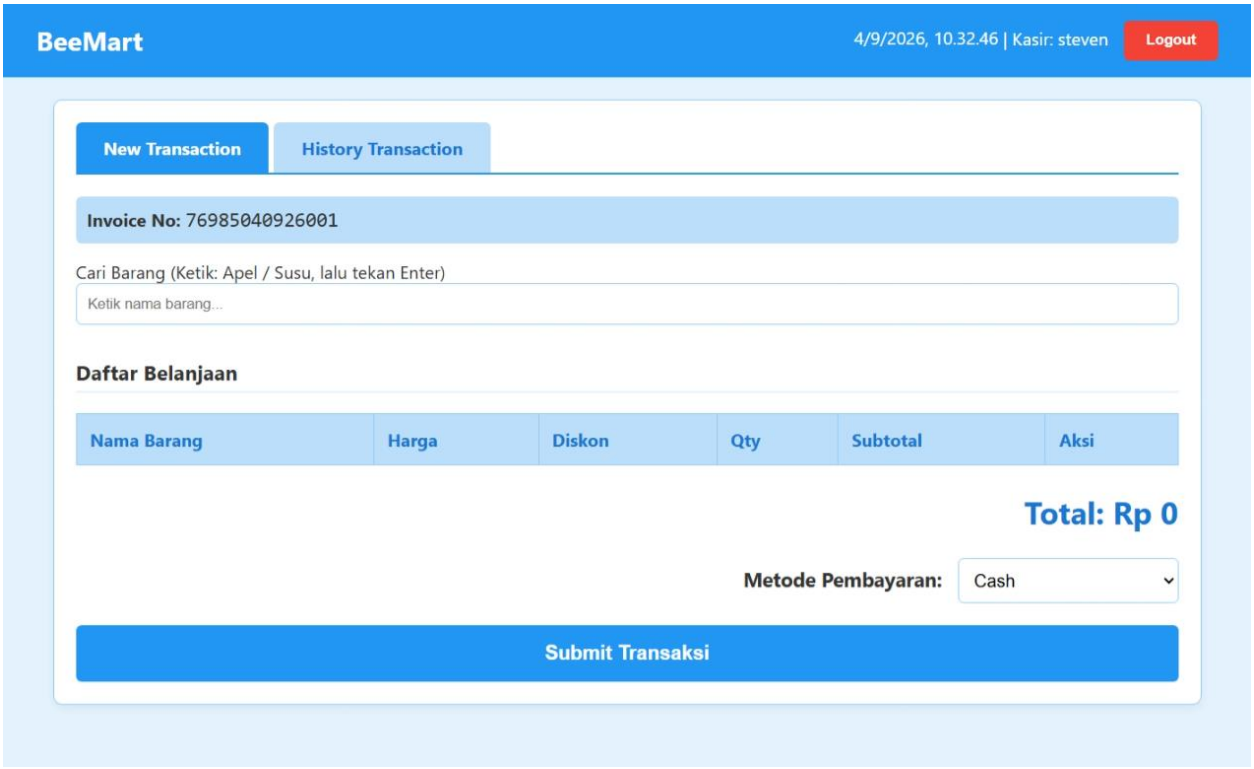
a. Username, Full Name, Password & Re-Password harus diisi
b. Password yang di simpan ke dalam database harus di hashing terlebih dahulu.
c. Username yang didaftarkan tidak boleh ada yang sama, jika ada yang sama return message error “username has already exists”.

3. New Transaction

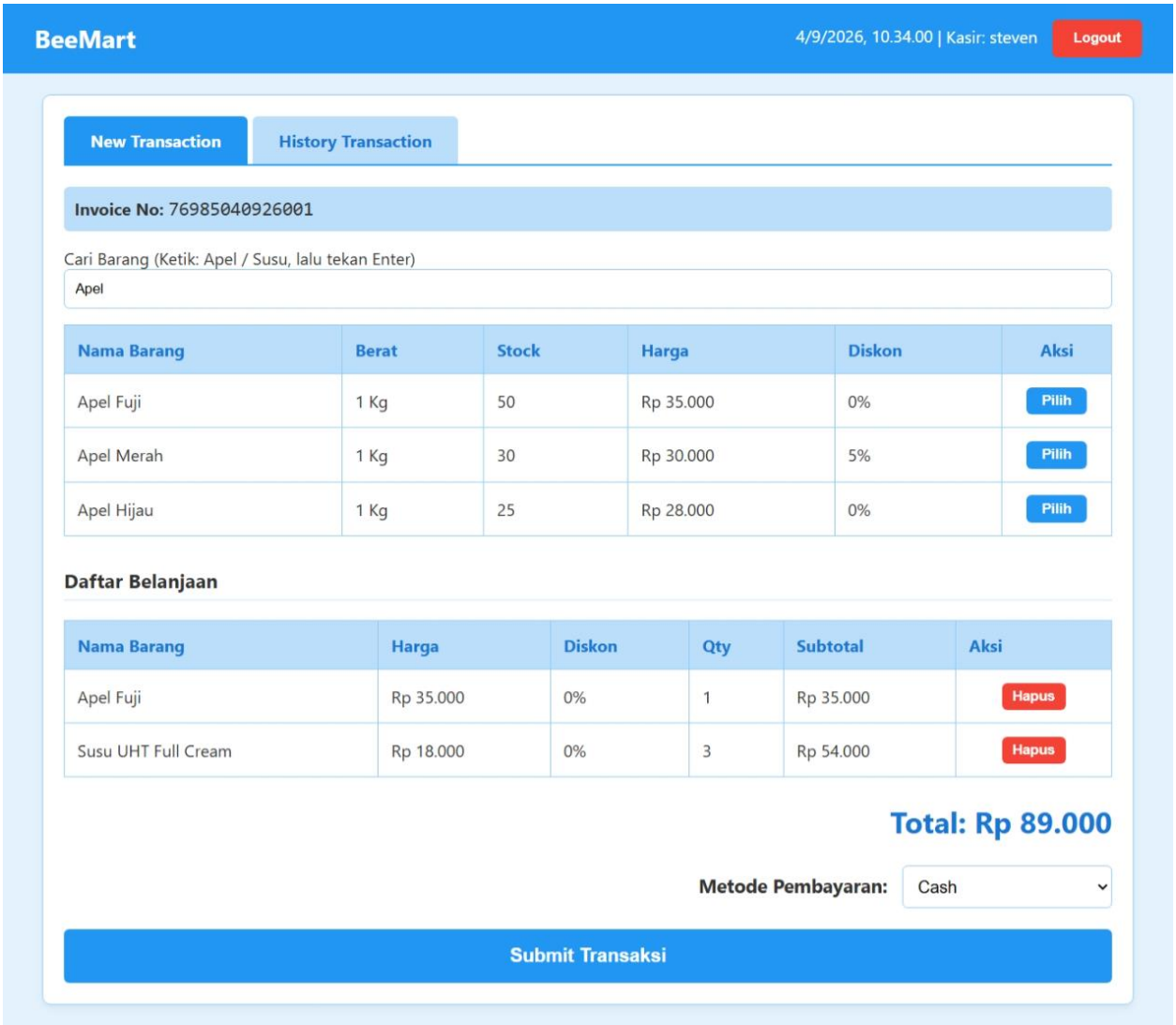
Pada menu ini bisa di akses role Cashier dan Admin, digunakan untuk melakukan transaksi pembelian barang.

Validasi yang harus diimplementasikan:

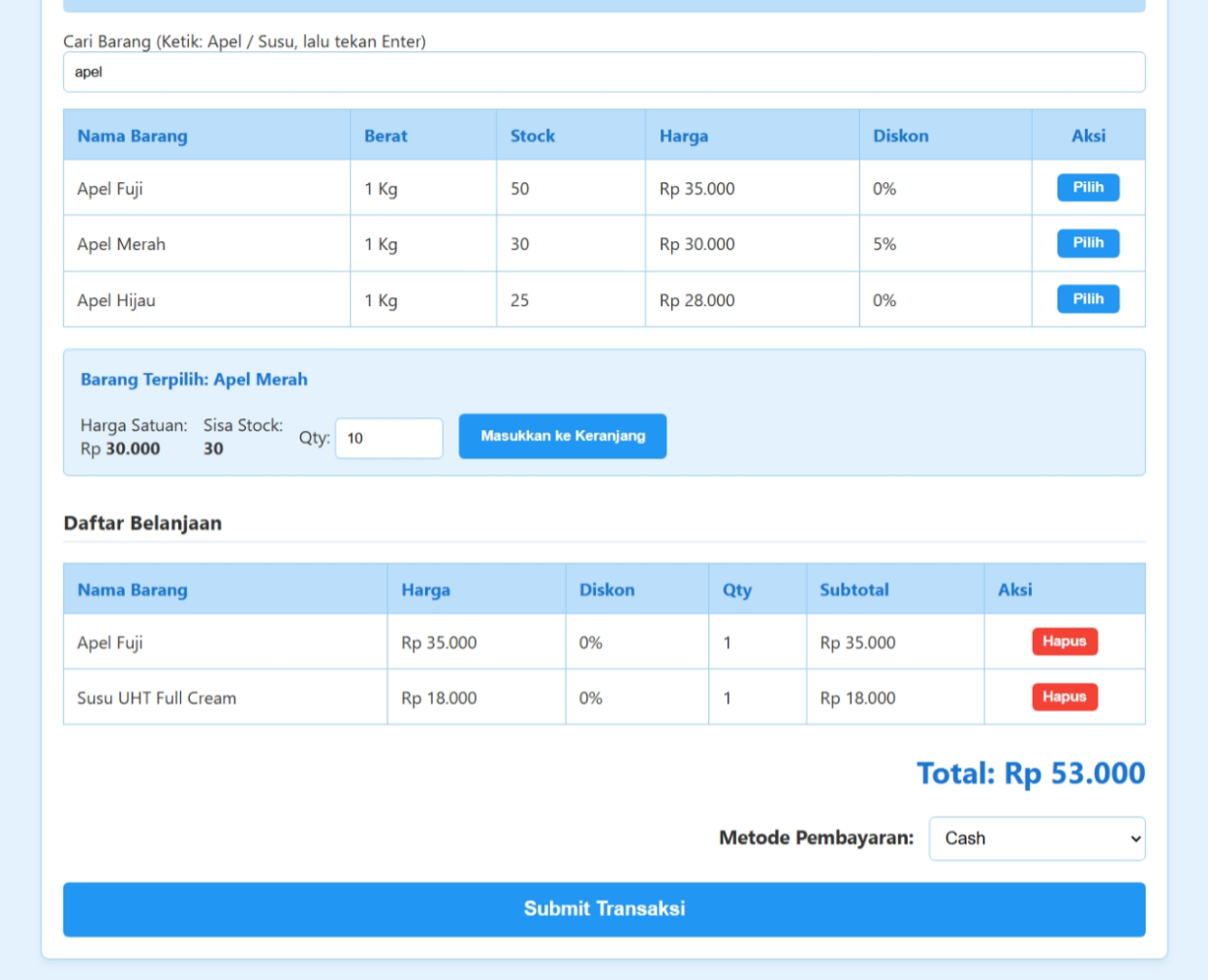
a. *Username*, *Full Name*, *Password*, dan *Re-Password* wajib diisi.
b. *Password* yang disimpan ke dalam *database* harus di-*hash* terlebih dahulu.
c. *Username* yang didaftarkan tidak boleh duplikat (harus unik). Jika *username* sudah terdaftar, sistem harus mengembalikan pesan *error* "Username already exists".



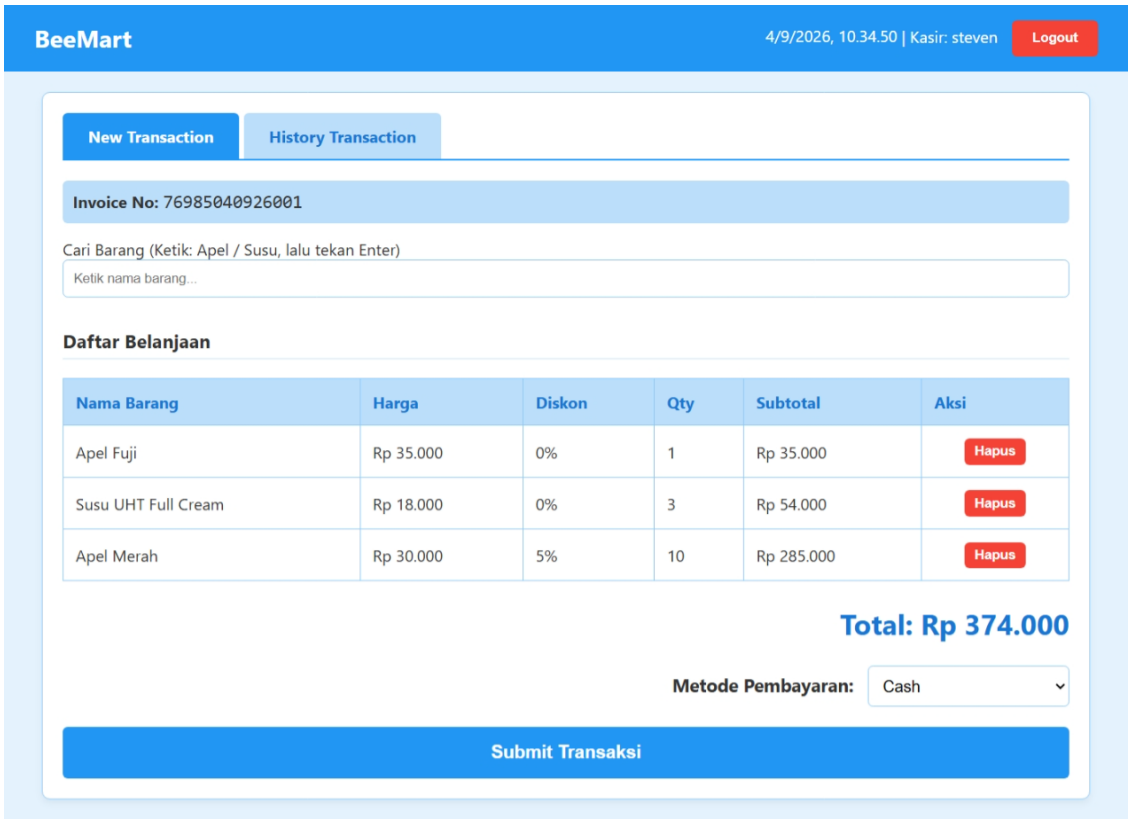
d. Fitur pencarian barang harus menampilkan maksimal 10 data dari *database* yang memiliki nama serupa dengan kata kunci yang dicari.



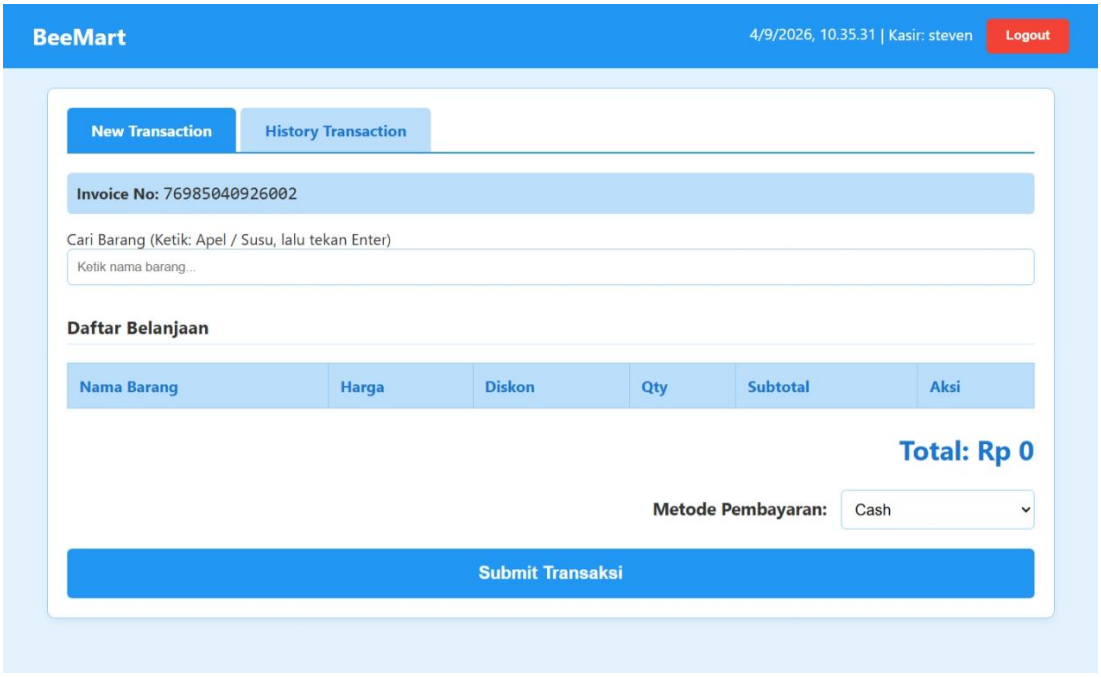
e. Pada tabel hasil pencarian, pengguna dapat memilih barang yang ingin ditambahkan dengan mengeklik tombol "Pilih".



f. Ketika tombol "Pilih" diklik, sistem akan menampilkan kotak dialog yang berisi informasi barang yang dipilih, harga satuan, dan sisa stok. Pengguna kemudian dapat memasukkan jumlah barang (*qty*) dan mengeklik tombol "Masukkan ke Keranjang". Jika jumlah barang yang diinput melebihi sisa stok, sistem akan menampilkan pesan *error*: "Jumlah barang yang dimasukkan melebihi sisa stok."
g. Pada bagian daftar belanjaan, pengguna dapat menghapus *item* atau barang yang sebelumnya telah ditambahkan.
h. Data pada *dropdown* Metode Pembayaran harus diambil (di-*fetch*) secara dinamis menggunakan API.



i. Jika proses *submit* transaksi berhasil, sistem akan mengatur ulang (*reset*) halaman transaksi ke kondisi awal dan menghasilkan *Invoice Number* baru untuk transaksi berikutnya.

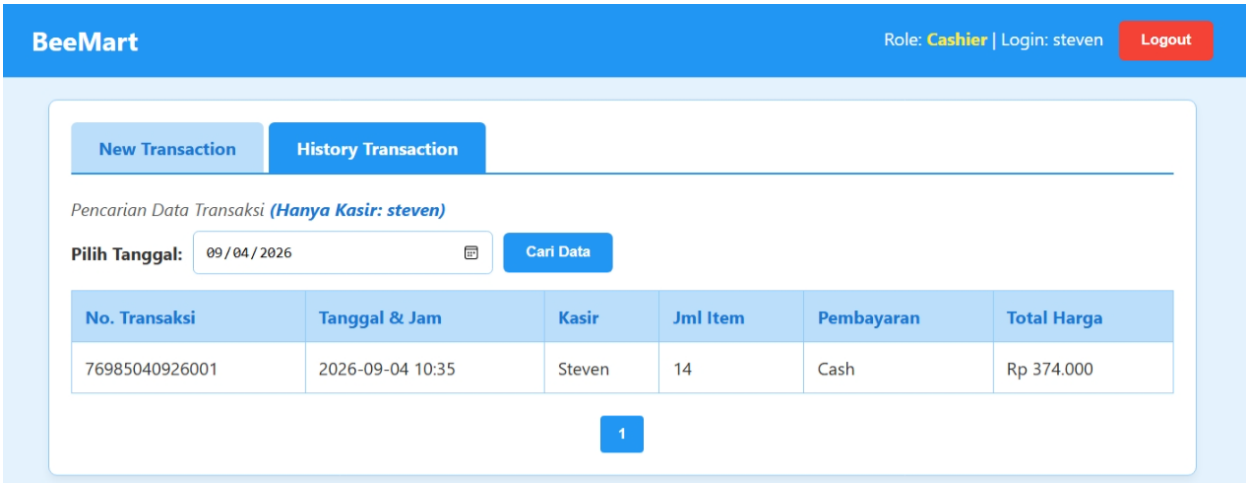


4. Riwayat Transaksi (Role: Kasir)

Halaman ini berfungsi untuk menampilkan data yang ditarik dari tabel transaksi berdasarkan tanggal yang telah dipilih oleh pengguna.

Ketentuan dan validasi yang harus diimplementasikan:

a. Untuk melakukan pencarian data, pengguna wajib memilih tanggal transaksi terlebih dahulu.
b. Input tanggal wajib menggunakan komponen *date picker*.
c. Tabel data transaksi harus mengimplementasikan fitur *pagination* dengan batas maksimal 20 data per halaman (*page*). Data tersebut harus diurutkan (*order by*) berdasarkan tanggal transaksi secara *ascending* (menaik).
d. Pada bagian *header* halaman, sistem harus menampilkan informasi sesi pengguna dengan format: `Role: <role user> | Login: <username user>`.



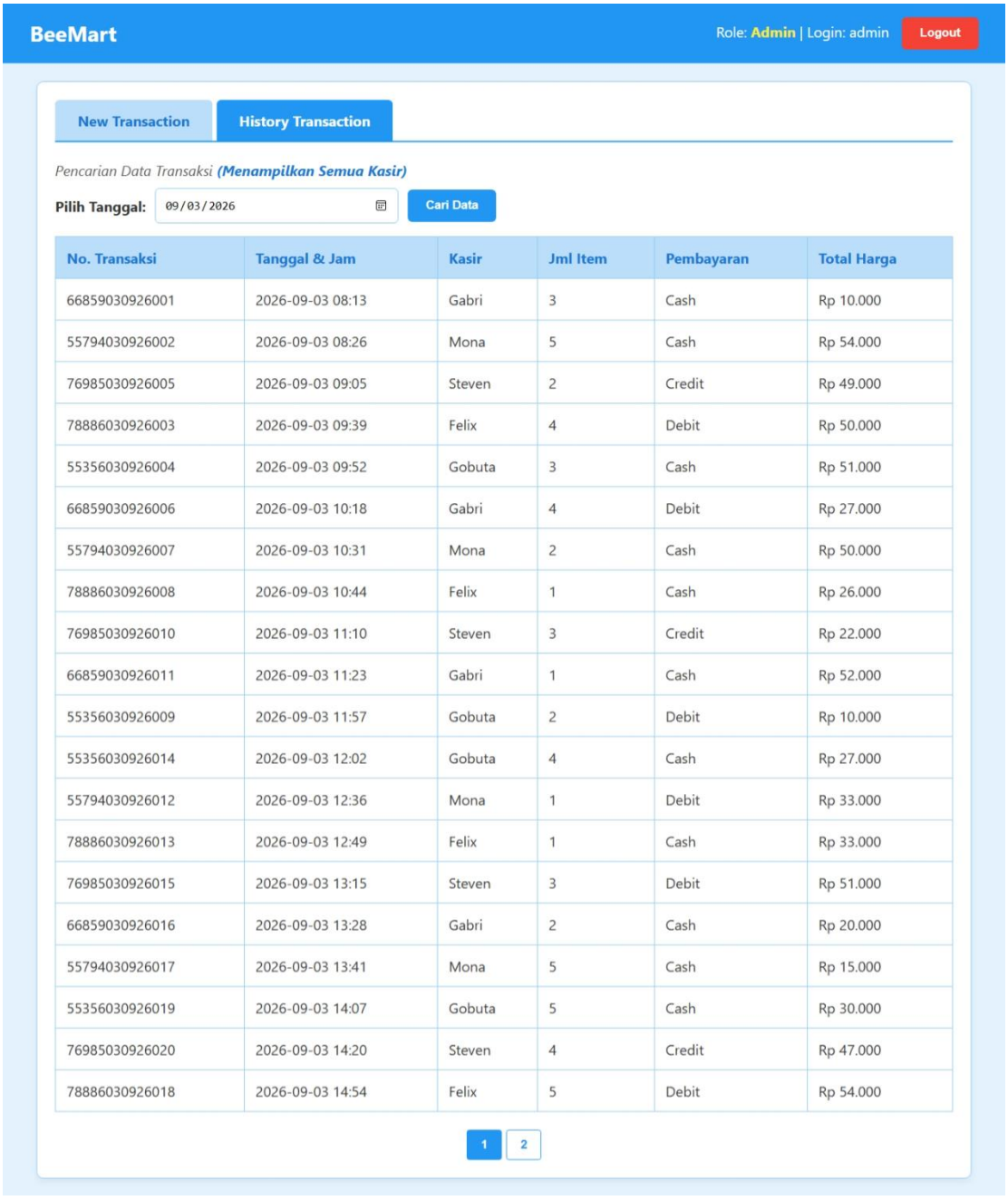
5. Riwayat Transaksi (Role: Admin)

Halaman ini berfungsi untuk menampilkan data yang ditarik dari tabel transaksi berdasarkan tanggal yang telah dipilih oleh pengguna.

Ketentuan dan validasi yang harus diimplementasikan:

a. Untuk melakukan pencarian data, pengguna wajib memilih tanggal transaksi terlebih dahulu.
b. Input tanggal wajib menggunakan komponen *date picker*.
c. Data yang dimunculkan harus mencakup seluruh riwayat transaksi yang dilakukan oleh semua kasir.

d. Tabel data transaksi harus mengimplementasikan fitur *pagination* dengan batas maksimal 20 data per halaman (*page*). Data tersebut harus diurutkan (*order by*) berdasarkan tanggal transaksi secara *ascending* (menaik).
e. Pada bagian *header* halaman, sistem harus menampilkan informasi sesi pengguna dengan format: Role: <role user> | Login: <username user>.



---GOOD LUCK---